

Suite au développement réussi du module de gestion des approvisionnements et des stocks pour l'entreprise Tricol, la direction informatique souhaite maintenant **sécuriser l'accès à cette application** avant sa mise en production.

L'application gère des données sensibles (informations fournisseurs, prix d'achat, valorisation des stocks) et doit être accessible à différents profils d'utilisateurs au sein de l'entreprise. Il est donc impératif de mettre en place un système d'authentification et d'autorisation robuste.

Objectif

Implémenter la sécurité de l'application en utilisant **Spring Security** afin de :

- Authentifier les utilisateurs
- Gérer les autorisations selon les rôles
- Protéger les endpoints de l'API REST
- Sécuriser les données sensibles

Utilisateurs et Rôles

L'application dispose de 4 rôles distincts. le fichier **Matrice des Permissions ci-joint pour le détail complet des autorisations par rôle.**

Gestion dynamique des permissions

Principe de fonctionnement

1. Inscription (Register) : Lorsqu'un nouvel utilisateur s'inscrit, il n'a aucun rôle attribué par défaut. Il ne peut donc effectuer aucune action dans l'application jusqu'à ce qu'un administrateur lui assigne un rôle.
2. Attribution de rôle par l'administrateur : L'administrateur peut assigner un rôle à un utilisateur (ex: MAGASINIER, CHEF_ATELIER, etc.). L'utilisateur hérite alors des permissions par défaut associées à ce rôle.
3. Personnalisation des permissions : L'administrateur peut modifier les permissions individuelles d'un utilisateur à tout moment, indépendamment de son rôle de base.

Exemple concret

L'utilisateur Amine a le rôle MAGASINIER. Par défaut, selon la matrice des permissions, il peut créer des bons de sortie. Cependant, l'administrateur

décide de lui retirer cette permission spécifique. Amine conserve son rôle MAGASINIER mais ne peut plus créer de bons de sortie, tout en gardant ses autres permissions (réceptionner les commandes, consulter le stock, etc.).

Fonctionnalités à implémenter

- **Configurer Spring Security** avec authentification JWT
- **Créer les entités:**
 - UserApp: Entité utilisateur avec informations de connexion
 - RoleApp: Entité rôle (ADMIN, RESPONSABLE_ACHATS, MAGASINIER, CHEF_ATELIER)
 - **Permission** : Entité permission
 - **UserPermission** : Table de liaison permettant de personnaliser les permissions par utilisateur
- **Implémenter les endpoints** d'authentification (login, register, gestion du refresh token)
- **Sécuriser les endpoints existants** selon la matrice des permissions
- **Système d'audit:**
 - Tracer les actions sensibles : qui a fait quoi et quand
 - Enregistrer les modifications de permissions
 - Journaliser les connexions/déconnexions
 - traçant les actions sensibles (qui a fait quoi et quand)

Test unitaire:

- Développement des tests unitaires pour l'authentification

Dockerization de l'application:

- Création du Dockerfile
- Construction de l'image Docker
- Exécution du conteneur à partir de l'image créée
- pusher l'image créée vers Docker Hub

CI avec GITHUB ACTIONS (1 workflow):

- Build
- Docker Build & Push
- Configurer les Secrets GitHub